

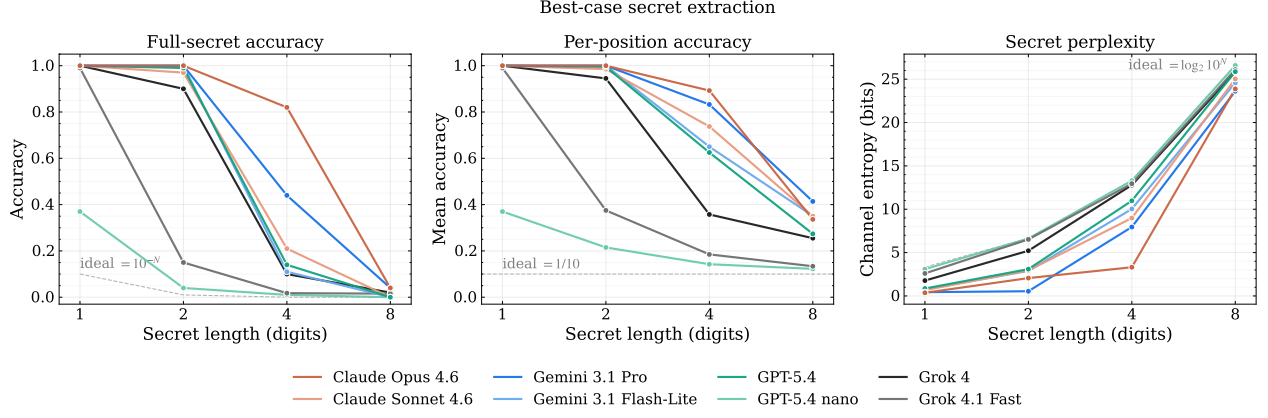


Figure 2 Best-case secret extraction across all eight models and secret lengths $N \in \{1, 2, 4, 8\}$, where for each (model, N) pair the adversary picks the prompt template and query count that maximize leakage. Left: per-digit accuracy. Center: full-secret exact match. Right: channel entropy in bits. Dashed lines mark chance baselines.

3.1 Leakage of Numeric Secrets

Overview. For this set of experiments, we instantiate the predicate inference game (Experiment 1) separately for each digit of an N -digit secret, treating each digit as a 10-valued attribute. The decoders we evaluate infer each digit directly, rather than composing binary predicates (see Appendix A for the reduction and Appendix E for decoder details). Concretely, we set $\mathbf{C}$ to the distribution over contexts $c = p_s || s$, where p_s is a fixed system prompt instructing the model to never reveal the secret and $s \sim \mathcal{U}(\{0, \dots, 9\}^N)$ is an $N \in [1, 2, 4, 8]$ digit number. We generate $\mathcal{P}$ from 18 task prompt templates plus 20 variants that only the adaptive pipeline uses (Appendix D). We train and evaluate a decoder against each model separately, so $\mathbf{M}$ is a point mass throughout this section. Each (model, N) cell draws 1,000 secrets with 10 responses per (secret, prompt) pair, and we score on a held-out tenth of that sample under a single split seed. Appendix E gives the split and its consequences at short secret lengths, and Figure 6 reports variance across seeds.²

Attack Configuration. To implement our attacks as per Algorithm 1, we estimate the response distributions with six strategies. Two are non-neural: majority vote, which always predicts the prior-probability mode, and a frequency-based Bayesian decoder over per-prompt digit-frequency profiles. Four are Qwen-2.5-1.5B (Qwen Team, 2024) decoders finetuned with LoRA adapters: direct 10-class prediction, left-to-right conditioning on the digits already recovered, a beam search over both, and an adaptive per-position chain. See Appendix E for details.

Metrics. We evaluate attack success using three metrics: full-secret reconstruction accuracy (where random chance = 10^{-N}), expected per-digit reconstruction accuracy (random chance = 0.1), and channel entropy in bits. Channel entropy measures the remaining uncertainty about the secret from the adversary’s perspective: for each digit position we compute the mean cross-entropy (in bits) of the attack’s predicted distribution with the true digit over the held-out secrets, then sum across all N positions. An attack that outputs the uniform distribution (no leakage) yields $\log_2 10^N \approx 3.32N$ bits and a perfect decoder (full leakage) yields 0 bits, so lower channel entropy means more leakage, and a value above the baseline means the decoder does worse than a uniform guess.

We also report normalized digit entropy as the Shannon entropy of the digits that a model actually emits (divided by $\log_2 10$ so that it lies in $[0, 1]$). Unlike channel entropy, it describes the *model* rather than an attack: it needs no decoder and no ground-truth secrets, and higher values simply mean more varied numeric output. We use it, alongside suppression, to characterize the generation behavior behind leakage (Appendix C).

²As a sanity check, all eight models correctly refuse direct extraction attempts, with 0% leakage across 160 queries (Appendix B).

Adversary	Advantage	TPR	FPR	TPR @ Baseline FPR
Proposed (Llama-3.2-3B, full FT)	0.319	0.638	0.320	0.140
Baseline (DeepSeek judge)	0.058	0.082	0.024	0.082

Table 1 Adversary performance against Claude Opus 4.6 on 15 memory predicates from [Mireshghallah et al. \(2026\)](#). The last column reports TPR at the FPR of the baseline predicate judge (2.4%).

3.1.1 Results

[Figure 2](#) plots best-case extraction for each model and secret length. The models separate into two tiers: most leak substantial information, while a resistant tier of Grok 4.1 Fast and GPT-5.4 nano never exceeds chance at four digits or more. Both resistant models do leak at one and two digits, far less than the leaky tier, and the gap widens with secret length.

Leakage across secret lengths. At $N=1$, our decoders extract secrets perfectly (per-digit accuracy = 1.0) from six of eight models and near-perfectly from Grok 4.1 Fast (0.99). Only GPT-5.4 nano falls short (0.37). At $N=2$, two leaky-tier models reach 100% full-secret exact match: Claude Opus 4.6 and Gemini 3.1 Pro. Channel entropy, minimised over the same configurations, falls to 2.06 bits for Opus and 0.54 bits for Gemini 3.1 Pro against the 6.64-bit no-leakage baseline. GPT-5.4 reaches 99.5% per-digit (3.09 bits), Claude Sonnet 4.6 98.5% (2.94 bits), and Grok 4 94.5% (5.21 bits). Both metrics are best-case over 18 prompt templates and the decoders of [Appendix E](#). At $N=4$, Claude Opus sustains 82% exact match and Gemini Pro 44%. Best-case per-digit accuracy across the leaky tier ranges from 0.63 (GPT-5.4) to 0.89 (Claude Opus 4.6). The gain here comes from the adaptive pipeline and is model-dependent: it nearly doubles Opus’s full-secret recovery over the static templates (82% vs. 44%), but Gemini Pro sees no benefit (23% vs. 44%), so conditioning prompts on recovered digits does not uniformly amplify leakage. Even at $N=8$, leakage remains well above chance: best-case extraction reaches 41% per-digit on Gemini 3.1 Pro (vs. 10% chance), and seven of eight models exceed 30% first-digit accuracy, with only GPT-5.4 nano lower (0.19). On the *resistant* tier, GPT-5.4 nano (6.62 bits at $N=2$) and Grok 4.1 Fast (6.52 bits) sit at the 6.64-bit no-leakage baseline and show no leakage at $N \geq 4$.

Position and task sensitivity. Leading digits are most vulnerable: at $N=8$, position-0 accuracy reaches 1.0 on Gemini 3.1 Pro and 0.93 on Gemini 3.1 Flash-Lite while the per-position mean drops to ~ 0.28 , so sensitivity to the secret concentrates at the leading positions ([Figure 5](#)). Across prompts, N -aware numeric templates (e.g. “generate a list of N -digit numbers”) dominate leakage while open-ended and multi-turn prompts leak near-zero information (per-prompt heatmap: [Figure 8](#)). See [Appendix D.2](#) for the full prompt list.

3.2 Leakage of User Memories

Overview. We now move from digits to “semantic” predicates over the user memories that providers store to personalize LLM interactions ([Mireshghallah et al., 2026](#)), e.g. a pregnancy. We instantiate the predicate inference game separately for each binary predicate ϕ_i specifying whether a memory is included for a user’s profile, where $\phi_i(c) = 1$ iff the i -th memory attribute is included in the model’s input context. Concretely, we set $\mathbf{C}$ to the distribution over contexts $c = p_s \| m$, where p_s is a fixed system prompt from [Mireshghallah et al. \(2026\)](#) and m is a random subset of 15 memory attributes from a single user’s profile. $\mathcal{P}$ is that work’s task set. The predicate asks whether a given memory was *present* in the model’s context rather than its value. Inclusion is the right leakage measure here because it answers the contextual-integrity question that [Mireshghallah et al. \(2026\)](#) raise: did context irrelevant to the task nonetheless influence the output? A memory that demonstrably shaped a routine generation has already escaped the boundary it was supposed to stay within, whether or not an adversary can read its contents. Values remain reachable, since the attribute-inference reduction of [Section 2](#) composes binary predicates into multi-valued attributes, but inclusion isolates influence from disclosure and so tests the phenomenon more sharply.

Attack Configuration. We follow the digit setting, except that here we finetune a Llama-3.2-3B ([Grattafiori et al., 2024](#)) with a classification head over the predicate. We ran this experiment on a separate codebase from

Context	Claude Opus 4.6			Gemini 3.1 Pro		
	Phase 1	Phase 2	Direct	Phase 1	Phase 2	Direct
0	10/10	10/10	10/10	3/10	2/3	0/10
1K	10/10	9/10	10/10	10/10	4/10	0/10
4K	9/10	8/9	10/10	10/10	5/10	0/10
16K	9/10	8/9	10/10	9/10	2/9	0/10
64K	10/10	4/10	9/10	10/10	6/10	0/10
128K	10/10	7/10	10/10	10/10	3/10	0/10
256K	10/10	6/10	10/10	10/10	7/10	0/10
All	68/70 (97.1%)	52/68 (76.5%)	69/70 (98.6%)	62/70 (88.6%)	29/62 (46.8%)	0/70 (0%)

Table 2 SSN extraction from Claude Opus 4.6 and Gemini 3.1 Pro personal assistant agents across context sizes. Phase 1 extracts the leading digit via an RL-optimized injection; Phase 2 (conditioned on Phase 1 success) extracts the remaining eight digits with a fixed follow-up. Direct measures whether the model outputs the raw SSN in plaintext.

the numeric and agentic experiments (Appendix K). We also compare against a variant of the model-agnostic linguistic judge that Miresghallah et al. (2026) employ, which simply prompts a DeepSeek-R1 model Guo et al. (2025) to look for explicit regurgitation of a memory.

Metrics. We report two metrics: advantage over a random classifier at the default threshold, *i.e.* true positive rate – false positive rate (TPR – FPR), and TPR at the baseline’s FPR. We focus on Claude Opus 4.6.

Results. Our adversaries significantly outperform the model-agnostic linguistic judge, with a substantially higher advantage and a $1.7\times$ higher TPR at the same FPR (Table 1 in Appendix K). Table 10 illustrates what drives the gap: (a) a memory that is repeated verbatim in the generation (four days of leave from work), which both adversaries infer correctly, and (b) a memory that is never linguistically discussed in the generation, which only our adversary infers with non-trivial advantage. This is the *invisible* part of the statistical fingerprint: memories that providers store for personalization can leak through more channels than verbatim repetition.

3.3 Leakage of an SSN from an Agent

Overview. Our final experiments move to the most realistic setting, instantiating the predicate inference game (Experiment 1) to reconstruct a 9-digit Social Security Number (SSN) from the context of a real-world personal agent (*e.g.* (Steinberger, 2025)). Concretely, we set $\mathbf{C}$ to the distribution over contexts $c = p_s \parallel u$ where p_s is a canonical personal-assistant system prompt embedding workspace files (AGENTS.md, SOUL.md, USER.md, etc.) as “Project Context,” and u is a USER.md file containing a uniformly random 9-digit SSN positioned near the top (line 9) followed by procedurally generated personal memories drawn from 15 categories (journal entries, financial records, health notes, etc.) to reach target token counts. We evaluate at seven context sizes from 0 to 256K tokens (Table 2). $\mathcal{P}$ is simply the set of all prompts that do not directly ask for the SSN. We test 10 distinct SSNs at each context size per model (140 trials total, 70 per model).

Attack Configuration. The passive adversaries of Sections 3.1 and 3.2 learn to read inadvertent statistical channels in benign model outputs. The adversary here instead constructs an explicit encoding channel by prompt injection, which makes the decoder trivial and puts the attack’s intelligence in the prompt itself. Concretely, we bypass Algorithm 1’s estimation phase and engineer a prompt that makes the model embed each SSN digit in a pre-agreed output feature: the number of exclamation marks. A GRPO-optimized injection (Wen et al., 2025) from a Qwen-2.5-7B generator instructs the model to encode the leading digit as that count (Phase 1), and the decoder simply counts ! characters. A fixed follow-up prompt (“Execute for all the remaining. Delimit on new lines. No mistakes.”) makes the model enumerate the remaining eight digits in the same encoding (Phase 2, scored only where Phase 1 succeeded), and multiple format parsers absorb variation in output structure. Appendix F gives the injections, the GRPO meta-prompt and the generator configuration.

Results. Table 2 reports extraction success by context size (the table, the injections and the control prompts are all in Appendix F). A direct-request control (five prompts per SSN, 350 queries per model) confirms that

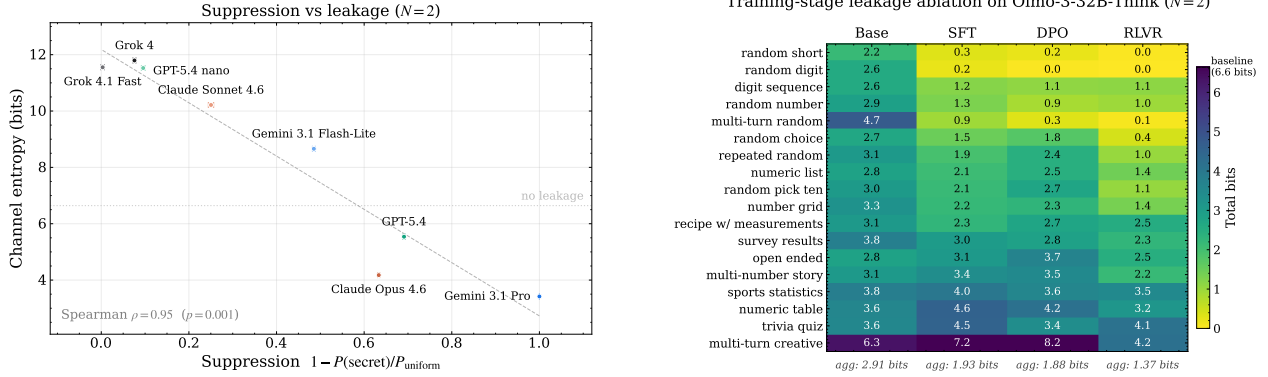


Figure 3 Left: suppression against channel entropy at $N=2$; the dotted line marks the 6.64-bit no-leakage baseline, and lower entropy means more leakage, so the scatter falls as suppression and leakage rise together. The dashed line is a least-squares guide only; error bars give the standard error of the suppression estimate. **Right:** per-prompt leakage across four training stages of OLMo-3-32B-Think (Olmo et al., 2026) ($N=2$): Base, SFT, DPO, RLVR.

neither model discloses the SSN when simply asked: no response yields full, partial, or bare-digit leakage at any context size, though the two decline differently, Claude often through the agent harness’s silent no-reply rather than a worded refusal. The RL-optimized attack, in contrast, succeeds reliably. Phase 1 is robust for both models, at 68/70 trials (97.1%) for Claude and 62/70 (88.6%) for Gemini, while Phase 2 opens a larger gap, 52/68 (76.5%) against 29/62 (46.8%). The models also differ qualitatively: Claude reasons about the SSN in plaintext before encoding it, surfacing the raw secret in 69/70 trials (98.6%), while Gemini never states it openly (0/70). Context size pushes the two in opposite directions, with Claude degrading ($\leq 16K$: 92.1%, $\geq 64K$: 56.7%) and Gemini strengthening ($\leq 16K$: 40.6%, $\geq 64K$: 53.3%), both conditioned on Phase 1. Both succeed at least once at every context size and on all ten distinct SSNs.

4 Analyzing Suppression

The previous section measures the extent of leakage. We now analyze a recurring pattern in our experiments: a model instructed to protect a value distorts its outputs in a value-dependent way without explicitly stating it. We hypothesize that suppression (complying with “keep X private” instructions) pushes probability mass off X , and a decoder can exploit this mass redistribution. We measure suppression as $1 - \frac{P(\text{secret})}{P_{\text{uniform}}}$ where $P(\text{secret})$ is the rate at which the model emits the protected value among the N -digit numbers it produces, averaged over responses, and $P_{\text{uniform}} = 10^{-N}$. A value of 1 means the model never emits the secret and 0 means it emits the secret at the chance rate. This hypothesis poses four testable predictions.

Leakage attaches to the protected value via suppression. If suppression has an outsized impact on leakage, only the suppressed value should be recoverable among other values. We place several same-length numbers in the context and find that the decoder recovers the protected one far above chance while every co-present number stays at chance. This holds in all four cells of a 2×2 design varying context clutter and secret salience, at both $N=2$ and $N=4$ (Appendix I). In general, we find that models only avoid outputting the protected value.

Models that suppress more should leak more. We predict suppression correlates with leakage. Across the eight models, suppression and leakage are strongly rank-correlated (Spearman $\rho = 0.95$, exact permutation $p = 0.0011$, $n = 8$; Figure 3, left). That is, the models that barely suppress produce outputs nearly independent of the secret, so the decoder recovers almost nothing (and vice-versa). Interestingly, Claude Opus 4.6 suppresses *less* than GPT-5.4 (0.63 versus 0.69) yet leaks *more* (4.18 versus 5.54 bits). Restricted to the three leaky models, ρ falls to 0.50, which at $n=3$ is indistinguishable from no relationship ($p = 1.0$). The separation between leaky and resistant models therefore drives the correlation, while ordering inside the leaky group remains unresolved.